FORMAT FOR PREPARATION OF PROJECT REPORT

FOR

Foundations of Artificial Intelligence

AI-DRIVEN FAKE NEWS DETECTION USING RULE-BASED REASONING

A PROJECT REPORT

Submitted by

2116230701406 – RAGAVENDHAN A

2116230701351 – NAVEEN ST



October, 2025

BONAFIDE CERTIFICATE

Certified that this project report "AI-Driven Fake News Detection Using Rule-Based Reasoning" is the bonafide work of "RAGAVENDHAN A, NAVEEN ST" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	9
3	PROPOSED SYSTEM	12
	3.1 ARCHITECTURE DIAGRAM	13
4	MODULES DESCRIPTION	19
	4.1 MODULE 1	19
	4.2 MODULE 2	20
	4.3 MODULE 3	21
	4.4 MODULE 4	22
5	IMPLEMENTATION AND RESULTS	23
	5.1 EXPERIMENTAL SETUP	23
	5.2 RESULTS	25
	5.3 TESTING AND VALIDATION	29
6	CONCLUSION AND FUTURE WORK	30
7	REFERENCES	33

ABSTRACT

The rapid growth of digital communication and social media has made news information easily accessible to people. However, the spread of misleading and unreliable information has also become a major concern. Fake news can influence public opinion, create confusion and cause social problems. Therefore, identifying suspicious news content and encouraging users to verify information before sharing it are important tasks.

This project, titled "AI-Driven Fake News Detection Using Rule-Based Reasoning," presents a web-based system called Veritas Fake News Detector. The system analyses user-submitted news text and its source using predefined rules and a knowledge base. It applies rule-based reasoning to identify suspicious patterns in the content.

The system examines source credibility, suspicious web addresses, sensational language, clickbait phrases, excessive capitalisation, unusual punctuation, unverified claims, supporting evidence, named attribution, urgency, conspiratorial language and absolute claims. Each activated rule contributes to or reduces a weighted suspicion score. The score is converted into a risk percentage and used to classify the submitted content into three categories: REAL, SUSPICIOUS and FAKE. The system also generates an explanation describing the indicators found during analysis.

The application is implemented using Python and the Flask web framework, with HTML and CSS used to create the user interface. A separate knowledge base and inference engine improve the organisation and maintainability of the system. The project demonstrates how rule-based reasoning, knowledge representation and inference can be applied to an Artificial Intelligence problem.

CHAPTER 1

INTRODUCTION

1.1 Introduction

News plays an important role in helping people understand current events, government decisions, science, education and social issues. Today, people receive news through websites, social media platforms, messaging applications and other digital sources. Although this makes information easily available, it also allows misleading information to spread quickly.

Fake news refers to false or misleading information presented in a way that may appear to be genuine news. Such content may contain exaggerated statements, unsupported claims, suspicious links or instructions encouraging people to share it immediately. Readers may find it difficult to distinguish reliable reporting from misleading content, especially when information is received through forwarded messages.

Artificial Intelligence provides several techniques for analysing information and supporting decision-making. One such technique is rule-based reasoning, in which predefined IF–THEN rules are used to evaluate input and derive conclusions.

The Veritas Fake News Detector applies rule-based reasoning to examine the characteristics of news text. It evaluates the submitted content against a collection of rules stored in a knowledge base. Based on the detected indicators, it calculates a suspicion score and generates a classification with an explanation.

1.2 Problem Statement

The widespread circulation of misleading news creates a need for a simple tool that can identify potentially unreliable content. Manual verification of every news item may take time, and users may not always recognise the warning signs of misleading information.

The objective is to develop a web-based system that analyses submitted news text and its source using predefined rules, calculates a risk score and presents an understandable result. The system should explain why certain indicators were detected instead of providing only a classification.

1.3 Objectives

The main objectives of the project are:

1. To develop a web application for analysing news text.
2. To create a knowledge base containing predefined indicators of suspicious and reliable reporting.
3. To implement a rule-based inference engine using Python.
4. To examine source credibility, language patterns, evidence and misleading framing.
5. To calculate a weighted suspicion score from the detected indicators.
6. To classify the submitted content as REAL, SUSPICIOUS or FAKE.
7. To provide explanations for the rules activated during analysis.

8. To demonstrate the application of Artificial Intelligence concepts without using machine learning.

1.4 Scope of the Project

The project focuses on the preliminary analysis of news articles, headlines and forwarded messages. The user can enter news text and optionally specify the source. The application examines the content using predefined patterns and produces an analysis report.

The system can be used for educational demonstrations, awareness about suspicious content and preliminary screening of news items. It is not designed to establish the actual truth of every claim. Its conclusions depend on the rules and vocabulary available in the knowledge base.

1.5 Need for the Project

The need for this project arises from the following factors:

- Rapid information sharing: News and forwarded messages can reach many people in a short time.
- Difficulty in verification: Readers may not have enough time or resources to verify every claim.
- Suspicious language: Sensational headlines and exaggerated claims may encourage readers to react without checking the facts.
- Lack of supporting evidence: Some content does not provide identifiable sources, studies or official statements.

- Need for explainability: Users should understand which characteristics influenced a system's decision.

The proposed system addresses these needs by providing a transparent, rule-based analysis of submitted content.

1.6 Technologies Used

Technology	Purpose
Python	Implements the application logic and inference engine
Flask	Handles web requests and application routes
HTML	Structures the web pages
CSS	Styles the user interface
Regular expressions	Identifies text patterns, URLs and formatting indicators
Knowledge base	Stores trusted sources, suspicious phrases and evidence markers
Rule-based inference	Evaluates the content and produces findings

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Fake news detection has become an important area of research because digital platforms allow information to spread rapidly. Different approaches have been proposed to identify misinformation, including manual fact-checking, source verification, linguistic analysis, machine learning and knowledge-based reasoning

2.2 Manual Fact – Checking

Manual fact-checking involves examining claims, checking the credibility of the publisher and comparing information with reliable sources. This approach can provide detailed verification when performed carefully. However, it may require significant time and human effort, especially when the amount of content is large.

The proposed project takes inspiration from the basic verification process by checking source information, supporting evidence and identifiable attribution.

2.3 Machine Learning-Based Approaches

Machine learning-based systems learn patterns from training data. They may use labelled examples of genuine and fake news to identify similarities in language or other features. Their performance depends on factors such as the quality of the dataset, feature representation and evaluation method.

Although machine learning is widely used in fake news research, it requires a different implementation approach from the one selected for this project. Veritas uses predefined rules rather than training a predictive model.

2.4 Rule-Based Reasoning

Rule-based reasoning is an Artificial Intelligence technique in which knowledge is represented through predefined rules. These rules commonly follow the structure IF a condition is satisfied, THEN perform an action or derive a conclusion.

For example:

- IF a news item contains several sensational expressions, THEN increase the suspicion score.
- IF the source is recognised as trusted, THEN reduce the suspicion score.
- IF multiple supporting references are detected, THEN provide a mitigating adjustment.

2.5 Knowledge-Based Systems

A knowledge-based system stores information about a particular domain and uses reasoning procedures to solve problems. Its main components generally include a knowledge base and an inference mechanism.

In this project, the knowledge base contains lists of source names, suspicious expressions, evidence indicators and other language patterns. The inference engine evaluates the input against the rules and generates findings. The final output is based on the combined score of those findings.

2.6 Research Gap and Proposed Approach

A basic keyword checker may identify suspicious words but fail to consider other factors, such as source credibility or the presence of supporting evidence. It may also treat every suspicious indicator as proof of falsehood.

The proposed system improves the basic approach by using multiple categories of rules, weighted positive and negative scoring, risk thresholds and explanations. Trusted-source indicators and supporting evidence can reduce the score, while suspicious patterns can increase it.

However, the system remains a heuristic screening tool. It does not independently verify claims against live news databases or determine factual truth through external evidence.

CHAPTER 3
PROPOSED SYSTEM

3.0 Overview of the Proposed System

The proposed system, Veritas Fake News Detector, is a web-based application developed using Python and Flask. It accepts news text and an optional source name from the user. The submitted content is analysed by a rule-based inference engine that uses predefined knowledge stored in a separate knowledge base.

The system does not use machine learning, neural networks or a trained classification model. Instead, it evaluates the input against twelve predefined rules. Each rule checks a particular characteristic of the news content and produces a finding when its condition is satisfied.

The findings are combined to calculate a risk percentage. The application then assigns a verdict and displays the detected indicators to the user.

The application has a modular structure, making it easy to maintain and update the predefined rules. It displays results quickly and explains the indicators behind each verdict. However, the results depend on the predefined rules and should be verified using reliable sources.

3.1 Architecture Diagram



Figure 3.1: Architecture of the Fake News Detector.

3.2 Working Principle

The system operates through the following steps:

1. The user enters the news text and optionally selects or enters the source name.
2. The application checks whether the input is empty and limits the text length to 20,000 characters.
3. The text is preprocessed to normalise whitespace, identify words, extract URLs and examine capitalisation and punctuation.
4. The inference engine evaluates the input using the predefined rules in the knowledge base.
5. Each activated rule produces a finding containing a rule name, category, explanation and score.
6. Positive scores increase suspicion, while negative scores reduce suspicion when reliable indicators are detected.
7. The combined score is converted into a risk percentage and compared with the classification thresholds.
8. The web application displays the verdict and an explanation of the findings.

3.3 Rule-Based Reasoning

The inference engine uses twelve rules grouped into three main categories: Source, Language, Evidence and Framing. Each rule evaluates a particular feature of the submitted news.

The following table summarises the rules implemented in the project.

Rule ID	Rule name	Purpose
R01	Source credibility	Checks the source against known trusted and unreliable source lists
R02	Suspicious web address	Identifies URL patterns associated with suspicious or imitation-style domains
R03	Sensational language	Detects emotionally loaded words
R04	Clickbait construction	Identifies curiosity-based headline phrases
R05	Excessive capitalisation	Detects unusually high use of uppercase words
R06	Excessive punctuation	Checks repeated or excessive question and exclamation marks
R07	Unverified claims	Identifies vague attributions such as “sources say”
R08	Supporting evidence	Checks for references, statistics, studies and other evidence indicators

R09	Named attribution	Identifies statements attributed to named individuals
R10	Urgency and virality pressure	Detects instructions to share content immediately
R11	Conspiratorial framing	Identifies predefined conspiracy-related expressions
R12	Absolute claims	Detects unsupported guarantees and absolute statements

3.4 Scoring and Classification

Each rule can contribute a positive or negative score. Positive scores indicate suspicious characteristics, whereas negative scores represent mitigating indicators.

The risk percentage is calculated by normalising the positive net score against a configured ceiling of 55.

$$\text{Risk Percentage} = \min\left(100, \text{round}\left(\frac{\max(0, S)}{55} \times 100\right)\right)$$

Here, (S) represents the combined weighted score from the activated rules.

The classification thresholds are:

Risk percentage	Classification	Meaning
0–24%	REAL	No significant red flags found by the configured rules
25–59%	SUSPICIOUS	Indicators require further verification
60–100%	FAKE	Strong indicators of unreliable content were detected

These categories represent the system's rule-based assessment. They do not establish the factual truth of a news item.

3.5 Advantages of the Proposed System

- It does not require a training dataset or model training.
- Its rules and scoring mechanism are understandable.
- The findings explain why a particular indicator was detected.
- The knowledge base and inference engine are maintained separately.
- New rules and vocabulary can be added to extend the system.
- Positive and negative scoring allows reliable indicators to reduce suspicion.
- The web interface makes the system easy to demonstrate and use.

3.6 Limitations of the Proposed System

- The quality of the results depends on the rules and vocabulary defined in advance.
- A false claim that avoids suspicious language may not be detected.
- A genuine article containing sensational language may receive a high risk score.
- Source credibility depends on the completeness and accuracy of the configured source lists.
- The system does not independently verify news claims against live databases.
- Short inputs may not provide sufficient information for meaningful analysis.

CHAPTER 4
MODULES DESCRIPTION

4.1 Module 1 – Text Preprocessing and Feature Extraction

This module prepares the user-submitted news content for analysis. It is implemented in the rule engine using the Analysis data structure.

The submitted text is stripped of unnecessary leading and trailing spaces, and repeated whitespace is normalised. A lowercase representation is created for case-insensitive pattern matching. The module also identifies words, counts the total number of words, extracts URLs and counts question marks and exclamation marks.

The module identifies fully capitalised words to detect possible shouting or attention-seeking language. Two-letter acronyms such as “US” and “UK” are excluded from the excessive-capitalisation check to reduce unnecessary warnings.

Regular expressions are used to recognise URLs and text patterns. Word-boundary matching is applied to reduce incorrect matches in which a suspicious word appears inside an unrelated word.

The processed information is passed to the inference engine for further evaluation.

4.2 Module 2 – Knowledge Base and Rule-Based Inference Engine

This module is the main reasoning component of the system. It uses the knowledge base stored in knowledge_base.py and the rule evaluation logic implemented in rules_engine.py.

The knowledge base contains predefined information, including:

- Trusted source names.
- Known unreliable publisher names.
- Sensational words and clickbait phrases.
- Indicators of unverified claims.
- Supporting evidence markers.
- Urgency and virality phrases.
- Conspiracy-related expressions.
- Absolute or exaggerated claims.
- Suspicious web-address patterns and classification settings.

The inference engine compares the preprocessed content with these predefined indicators. When a rule condition is satisfied, the engine generates a finding with a rule identifier, category, explanation and score.

For example, if the submitted text contains a predefined clickbait phrase, the clickbait rule is activated. If the article contains a recognised trusted source or several supporting evidence indicators, mitigating findings may reduce the overall score.

The rules are registered individually, making the system modular and allowing new rules to be added without rewriting the entire inference process.

4.3 Module 3 – Suspicion Scoring and Classification

This module combines the findings produced by the inference engine.

Suspicious indicators contribute positive points, while mitigating indicators contribute negative points. The combined score is normalised into a percentage between 0 and 100. Negative net scores are treated as zero risk for the purpose of this calculation.

The resulting percentage is compared with the predefined thresholds. Scores below 25% are classified as REAL, scores from 25% to 59% as SUSPICIOUS, and scores of 60% or higher as FAKE.

The system also assesses confidence based on the amount of text and the number of findings. Very short submissions receive low confidence because the available information may be insufficient for a meaningful assessment.

This module ensures that the final output is based on the combined indicators rather than a single keyword alone.

4.4 Module 4 – Web Interface and Report Generation

This module provides the interface through which users interact with the application. It is implemented using Flask, HTML templates and CSS.

The home page accepts the news text and source name. When the user submits the form, Flask passes the information to the inference engine and stores the resulting analysis in the session.

The report page displays the verdict, risk percentage, confidence level and findings produced by the rules. A separate knowledge page allows users to inspect the configured rules and knowledge-base vocabulary.

The application also provides a JSON API endpoint for programmatic analysis. Input validation and error-handling routes are included to handle empty submissions and common web errors.

The user interface and explanation report make the system suitable for demonstration and educational use.

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The Veritas Fake News Detector was implemented as a Python-based web application. Flask was used to manage HTTP requests, render HTML templates and connect the user interface with the inference engine.

The project is organised into separate files for the application, knowledge base, rule engine, web templates, CSS and automated tests. This modular structure helps separate the interface, stored knowledge and reasoning logic.

Software Requirements

Component	Requirement
Programming language	Python
Web framework	Flask
Frontend	HTML and CSS
Pattern analysis	Python regular expressions
Browser	Any modern web browser
Development environment	Python-compatible code editor or terminal
Dependency management	pip and requirements.txt
Testing	Python test script

Main Project Files

File or folder	Description
app.py	Flask application, routes and input handling
knowledge_base.py	Stores source lists, phrase patterns and scoring thresholds
rules_engine.py	Implements preprocessing, inference, scoring and classification
templates/index.html	Provides the news input form
templates/report.html	Displays the analysis report
templates/knowledge.html	Displays the configured knowledge base and rules
templates/base.html	Provides the common web page structure
static/style.css	Defines the application's visual appearance
tests/test_rules.py	Contains automated rule tests
TEST_EXAMPLES.md	Documents example inputs and their expected results
requirements.txt	Specifies the Flask dependency

Execution Procedure

The application can be executed using the following steps:

1. Install Python and ensure that it is available in the system PATH.
2. Open a terminal in the project directory.
3. Install the required dependency using pip install -r requirements.txt.

4. Start the application by running python app.py.
5. Open a browser and navigate to http://127.0.0.1:5000.
6. Enter a news article or forwarded message and select or enter its source.
7. Submit the input to view the analysis report.
8. Open the knowledge page to inspect the rules and predefined indicators.

5.2 Results

The implemented system was examined using example inputs covering different types of news content. The examples were designed to demonstrate how the system responds to suspicious language, missing evidence, source credibility, urgency and reliable reporting indicators.

The following cases are documented in the project's test examples.

Case 1: Highly Suspicious Content

Input characteristics: Sensational expressions, clickbait phrases, suspicious links, conspiracy-related language, unsupported claims and requests to share the message.

Source: Unknown

Observed result: FAKE

Risk percentage: 100%

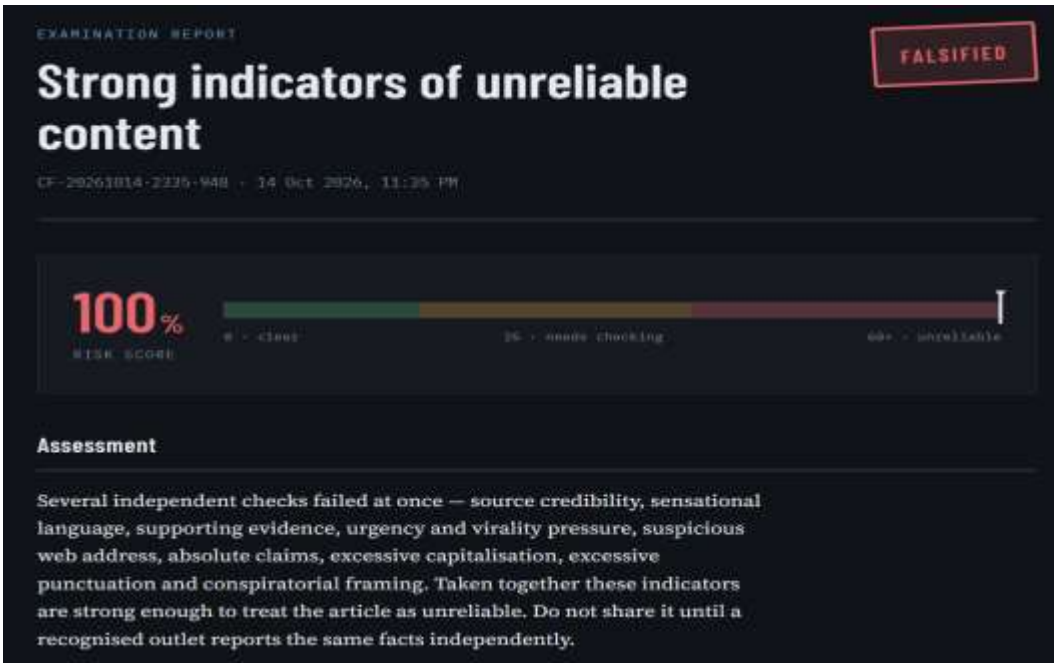


Figure 5.2.0: Falsified News

Explanation: Multiple suspicious rules were activated, including rules for sensational language, suspicious URLs, unverified claims, urgency and absolute claims. Their combined weighted score reached the configured maximum risk percentage.

Case 2: Plausible but Unsupported News

Input characteristics: A local-government claim attributed to unnamed sources, without an official notification or supporting references.

Source: Local Herald

Observed result: SUSPICIOUS

Risk percentage: 40%

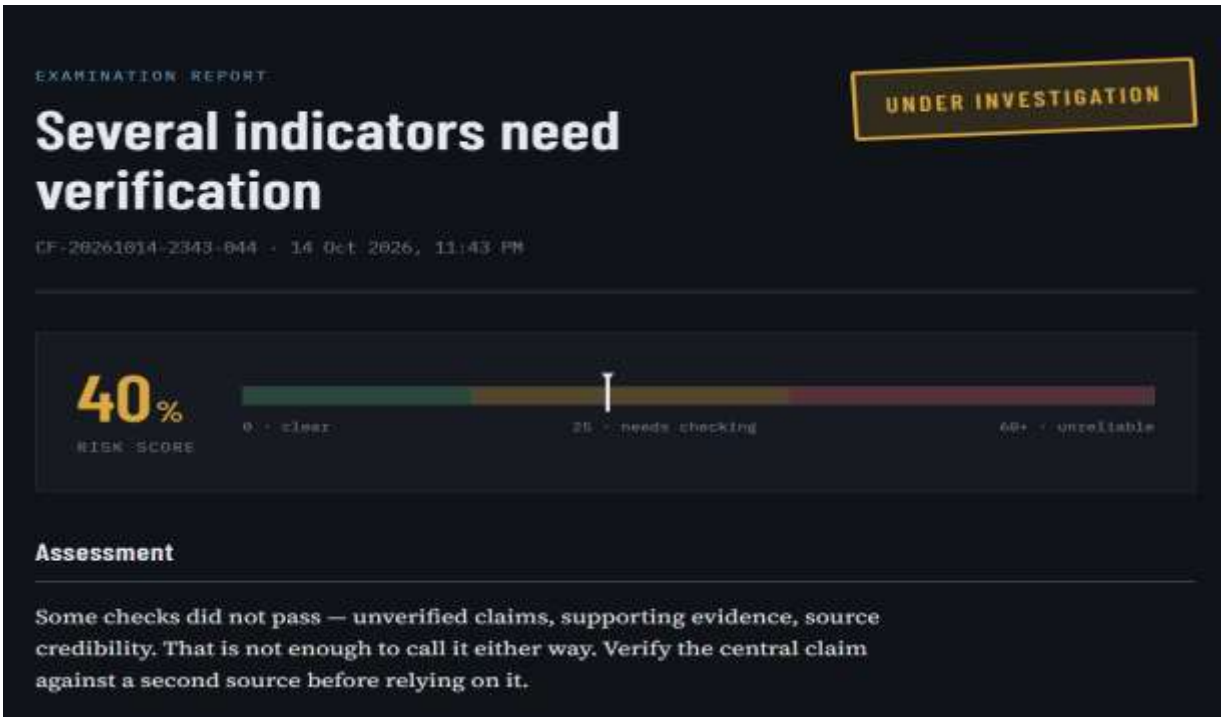


Figure 5.2.1: Suspicious News

Explanation: The system identified an unrecognised source, vague attribution and a lack of supporting evidence. The result indicates that the claim requires further verification; it does not establish that the claim is false.

Case 3: News with Supporting Evidence

Input characteristics: A report containing a named source, a reported statistic, an official bulletin and a research reference.

Source: The Hindu

Observed result: REAL

Risk percentage: 0%



Figure 5.2.2: No Red Flags (Real News)

Explanation: The trusted-source rule and evidence-related rules generated mitigating scores. The named attribution rule also reduced suspicion, resulting in a final risk percentage of zero.

Summary of Results

Test case	Main indicators	Risk	Classification
Case 1	Multiple suspicious indicators	100%	FAKE
Case 2	Unverified claims and missing evidence	40%	SUSPICIOUS
Case 3	Trusted source and supporting evidence	0%	REAL

These examples demonstrate that the system can produce different classifications based on the combination of detected indicators. They are illustrative tests of the implemented rules, not a measurement of real-world detection accuracy.

5.3 Testing and Validation

The project includes automated tests for rule behaviour and important edge cases. The tests help verify that the inference engine applies its conditions consistently.

The following aspects are covered by the project's testing examples:

1. Source credibility checks for trusted, unreliable and unrecognised sources.
2. Detection of sensational language and clickbait phrases.
3. Detection of excessive capitalisation and punctuation.
4. Identification of vague claims and missing evidence.
5. Recognition of named attribution and mitigating indicators.
6. Handling of suspicious links that should not be treated as supporting evidence.
7. Handling of very short text with low confidence.
8. Avoidance of incorrect matches inside unrelated words.
9. Classification based on the configured risk thresholds.
10. Detection of absolute claims that should not automatically count as statistical evidence.

The test examples also demonstrate that changing the source while keeping the news text unchanged can affect the final result. This illustrates the role of source credibility in the rule-based scoring process.

The project does not use a labelled benchmark dataset to measure precision, recall, F1-score or overall classification accuracy. Therefore, the results should be interpreted as functional demonstrations of the implemented rules rather than proof of general detection performance.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The project, “AI-Driven Fake News Detection Using Rule-Based Reasoning,” successfully demonstrates the application of knowledge representation and rule-based inference to the preliminary analysis of news content.

The Veritas Fake News Detector was developed using Python and Flask, with a separate knowledge base and inference engine. The system analyses submitted text using twelve predefined rules that cover source credibility, language patterns, supporting evidence and misleading framing.

The activated rules contribute positive or negative scores, which are combined to calculate a risk percentage. Based on the configured thresholds, the system classifies the content as REAL, SUSPICIOUS or FAKE. It also provides explanations for the detected indicators, making its decisions easier to understand.

The project demonstrates several important Artificial Intelligence concepts, including knowledge representation, inference rules, weighted scoring and explainable decision-making. Its modular design also allows the knowledge base and rules to be extended in the future.

However, the system does not independently verify factual claims and may produce incorrect classifications when the input contains misleading patterns or when relevant information is absent from the knowledge base. Its output should therefore

be treated as a preliminary assessment rather than a final judgment about the truth of a news item.

Overall, the project provides a practical demonstration of how a rule-based Artificial Intelligence system can support awareness and preliminary screening of potentially unreliable news.

6.2 Future Work

The following improvements could be considered in future versions:

1. Expanded knowledge base: Add more verified sources, unreliable publisher indicators and language patterns.
2. External fact-checking integration: Connect the application to reliable fact-checking services or official information sources to support claim verification.
3. Multilingual analysis: Extend the system to handle regional languages and mixed-language messages.
4. Improved semantic analysis: Examine the meaning and context of claims rather than relying only on predefined words and phrases.
5. Machine learning integration: Explore a hybrid approach combining explicit rules with a trained model, while retaining explanations for rule-based findings.
6. User feedback: Allow users to report incorrect classifications and use reviewed feedback to improve the knowledge base.
7. Better source verification: Use updated source information and more detailed checks of website identity and reliability.

8. Performance evaluation: Test the system on a suitable labelled dataset and calculate precision, recall, F1-score and other evaluation metrics.
9. Security improvements: Strengthen configuration management, session security and deployment settings before making the application publicly accessible.
10. Improved reporting: Provide more detailed explanations and links to trustworthy verification resources where available.

These enhancements could make the system more comprehensive and useful while preserving the transparency of its rule-based reasoning.

CHAPTER 7

REFERENCES

1. Flask Documentation. (n.d.). *Flask Documentation*. Pallets Projects. Available at: https://flask.palletsprojects.com/
2. Jackson, P. (1998). *Introduction to Expert Systems* (3rd ed.). Addison-Wesley.
3. Giarratano, J. and Riley, G. (2005). *Expert Systems: Principles and Programming* (4th ed.). Thomson Course Technology.
4. Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
5. Shu, K., Sliva, A., Wang, S., Tang, J. and Liu, H. (2017). Fake News Detection on Social Media: A Data Mining Perspective. *ACM SIGKDD Explorations Newsletter*, 19(1), pp. 22–36. Available at: https://doi.org/10.1145/3137597.3137600
6. Vosoughi, S., Roy, D. and Aral, S. (2018). The spread of true and false news online. *Science*, 359(6380), pp. 1146–1151. Available at: https://doi.org/10.1126/science.aap9559
7. Wardle, C. and Derakhshan, H. (2017). *Information Disorder: Toward an Interdisciplinary Framework for Research and Policy Making*. Council of Europe. Available at: https://rm.coe.int/information-disorder-toward-an-interdisciplinary-framework-for-researc/168076277c